

# 1. Files

As mentioned in the previous section<sup>1</sup>, on a Linux operating system everything is a file. A file is a structure that contains data and information about that data. In the C Library there are functions that allow you to read the information of the files and others that allow you to modify the properties of the files. Linux supports several types of files. For example, devices such as terminals, keyboards, and displays are represented in the filesystem using special device files, while ordinary text or data is stored in regular files. Each file in a Linux file system belongs to a particular user and group, has a creation date and a last modification date and has a set of permissions that define who and how can access the file and what operation can be performed on it (read, write, execute). A directory is a point of access in the file system and within it there can be files or directories. However a directory also is a file like everything else in a Linux fs.

## 1.1 Stat of files: `stat`, `fstat` and `lstat`

To retrieve the information about a file, the following functions are used.

```
#include <sys/stat.h>

int stat(const char *restrict pathname, struct stat *restrict statbuf);

int fstat(int fd, struct stat *statbuf);

int lstat(const char *restrict pathname, struct stat *restrict
          statbuf);

#include <fcntl.h>
#include <sys/stat.h>

int fstatat(int dirfd, const char *restrict pathname, struct stat
            *restrict statbuf, int flags);

Return: on success 0, on error -1 and errno
```

All these functions return information about a file through a buffer called `statbuf`. To access information about a file, a program must have permission to traverse every directory in the file's path. The required permission is execute permission, since traversing a directory requires the user or program to *execute* that directory.<sup>2</sup> `stat`, `fstatat` and `lstat` need these permissions to reach the file and read the metadata.

`stat` and `fstatat` retrieve the information about the file pointed to by `pathname`.

`lstat` is identical to `stat`, except that if `pathname` refers to a symbolic link, it returns information about the link itself rather than about the file to which the link points.

`fstat` is identical to `stat` except that the file is specified by a file descriptor (`fd`).

All these functions return information in a `stat`<sup>3</sup> structure (`statbuf`).

---

<sup>1</sup> Linux System Programming: Mastering File I/O

<sup>2</sup> To execute a directory means literally 'go inside the directory', to read a directory means 'read the content of the directory'

<sup>3</sup> See the `stat(3)` manual page

Since the purpose of these functions is to retrieve information about a file, the structure used to store the information and the pathname itself must each be accessed through a single pointer. For this reason, `pathname` and `statbuf` are declared with the `restrict`<sup>4</sup> qualifier. The `dirfd` argument of `fstatat` determines how the file path is resolved according to the following rules:

- if `pathname` is relative, it is interpreted relative to the directory specified in `dirfd` rather than relative to the current working directory
- if `pathname` is relative and `dirfd` has the special value `AT_FDCWD` then `pathname` is interpreted relative to the current working directory
- If `pathname` is an absolute pathname `dirfd` is ignored

The `flags` argument can be 0 or one of the following values:

|                                  |                                                                                                                                                                                                                                                                                   |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>AT_EMPTY_PATH</code>       | If <code>pathname</code> is an empty string the function will operate on the file referred to by <code>dirfd</code> . This last in this case can refer to any type of file. If <code>dirfd</code> is <code>AT_FDCWD</code> the function operate on the current working directory. |
| <code>AT_NO_AUTOMOUNT</code>     | Don't automount the terminal ("basename") component of <code>pathname</code>                                                                                                                                                                                                      |
| <code>AT_SYMLINK_NOFOLLOW</code> | If <code>pathname</code> is a symbolic link do not dereference it and return information on the link itself                                                                                                                                                                       |

**Tab 1.** Values of the `flags` argument of `fstatat`

The possible errors returned in `errno` are the following:

|                                |                                                                                                                                                                                |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>EACCESS</code>           | search permission is denied for one of the directories in the path                                                                                                             |
| <code>EBADF</code>             | <code>fd</code> is not a valid file descriptor                                                                                                                                 |
| <code>EBADF (fstatat)</code>   | <code>pathname</code> is relative but <code>dirfd</code> is not <code>AT_FDCWD</code> nor a valid descriptor                                                                   |
| <code>EFAULT</code>            | bad address                                                                                                                                                                    |
| <code>EINVAL (fstatat)</code>  | <code>flags</code> has not a valid value                                                                                                                                       |
| <code>ELOOP</code>             | too many symbolic links encountered while traversing the path                                                                                                                  |
| <code>ENAMETOOLONG</code>      | <code>pathname</code> is too long                                                                                                                                              |
| <code>ENOENT</code>            | a component of <code>pathname</code> does not exist or is a not valid symbolic link                                                                                            |
| <code>ENOENT (fstatat)</code>  | <code>pathname</code> is an empty string and <code>AT_EMPTY_PATH</code> is not specified                                                                                       |
| <code>ENOMEM</code>            | out of memeory                                                                                                                                                                 |
| <code>ENOTDIR</code>           | a component of <code>pathname</code> is not a directory                                                                                                                        |
| <code>ENOTDIR (fstatat)</code> | <code>pathname</code> is relative and <code>dirfd</code> is a file descriptor referring to a file other than a directtory                                                      |
| <code>EOVERFLOW</code>         | <code>pathname</code> or <code>fd</code> refers to a file whose size, inode number, or number of blocks cannot be represented in, respectively, the types <code>off_t</code> , |

4 About the use of the keyword `restrict` consult the *C/99 Standard*, the link is in the bibliography

|  |                              |
|--|------------------------------|
|  | ino_t, blkcnt_t <sup>5</sup> |
|--|------------------------------|

**Tab 2.** errno values for the stat functions

The information retrieved from the file is stored in a structure of type `stat`<sup>6</sup>. This structure represents the status of the file.

```
#include <sys/stat.h>

struct stat {
    dev_t st_dev;           /* ID of device containing file */
    ino_t st_ino;           /* Inode number */
    mode_t st_mode;         /* File type and mode */
    nlink_t st_nlink;       /* Number of hard links */
    uid_t st_uid;           /* User ID of owner */
    gid_t st_gid;           /* Group ID of owner */
    dev_t st_rdev;          /* Device ID (if special file) */
    off_t st_size;          /* Total size, in bytes */
    blksize_t st_blksize;   /* Block size for filesystem I/O */
    blkcnt_t st_blocks;     /* Number of 512B blocks allocated */

    /* Since POSIX.1-2008, this structure supports nanosecond
       precision for the following timestamp fields */

    struct timespec st_atim; /* Time of last access */
    struct timespec st_mtim; /* Time of last modification */
    struct timespec st_ctim; /* Time of last status change */

    #define st_atime st_atim.tv_sec /* Backward compatibility */
    #define st_mtime st_mtim.tv_sec
    #define st_ctime st_ctim.tv_sec
};
```

The `stat` structure is composed by the following fields:

|                       |                                           |
|-----------------------|-------------------------------------------|
| <code>st_dev</code>   | device on which the file resides          |
| <code>st_ino</code>   | the inode <sup>7</sup> number of the file |
| <code>st_mode</code>  | type and mode of the file                 |
| <code>st_nlink</code> | number of hard links to the file          |
| <code>st_uid</code>   | user ID of owner of the file              |
| <code>st_gid</code>   | group ID of the owner of the file         |
| <code>st_rdev</code>  | device represented by the file            |

5 See the `stat(2)` manual page for an example of this error and the manual pages: `off_t(3type)`, `stat(3type)`, `blkcnt_t(3type)` for the types

6 See the `stat(3type)` manual page

7 An inode is a number that identifies a file on the file system, each file and directory has his unique inode number

|            |                                                              |
|------------|--------------------------------------------------------------|
| st_size    | size of the file                                             |
| st_blksize | block size                                                   |
| st_blocks  | number of blocks allocated to the file (1 block = 512 bytes) |
| st_atime   | time of last access to the file data                         |
| st_mtime   | time of last modification of file data                       |
| st_ctime   | file's last status change timestamp                          |

**Tab 3.** Fields of the `stat` structure

The other fields (`st_atim.tv_sec`, `st_mtim.tv_sec`, and `st_ctim.tv_sec`) are provided for backward compatibility. The `st_ino` field represents the inode number of the file. An *inode* is a number that uniquely identifies a file within the filesystem. Every file and directory in a Linux filesystem has its own *inode*. On Linux systems, information about a file can be displayed using the `stat` program. When a symbolic link is created, the link stores a reference to the pathname of another file rather than sharing the same *inode*.

We create a file and write some data.

```
[linux@fidir] touch file.txt
[linux@fidir] echo 'hello' > file.txt
```

Then we create a symbolic link to the file with `ln` and the option `-s`

```
[linux@fidir] ln -s file.txt file2
[linux@fidir] ls -l
total 4
lrwxrwxrwx 1 fidir fidir 8 Mar 23 19:58 file2 -> file.txt
-rw-r--r-- 1 fidir fidir 6 Mar 23 19:58 file.txt
```

As we can see, the link `file2` point to the file `file.txt`. If we run `cat` on `file2` we see the content of `file.txt`

```
[linux@fidir] cat file2
hello
```

To retrieve information about the file we use the `stat` command.

```
[linux@fidir] stat file.txt
File: file.txt
Size: 6      Blocks: 8      IO Block: 4096  regular file
Device: 259,2 Inode: 23364606 Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/  fidir)  Gid: ( 1000/  fidir)
Access: 2024-03-23 20:01:37.452040888 +0700
Modify: 2024-03-23 19:58:18.396035269 +0700
Change: 2024-03-23 19:59:46.536037757 +0700
Birth: 2024-03-23 19:58:11.664035079 +0700
```

The output of `stat` shows detailed file metadata, including file size, number of blocks used, file type, device ID, inode number, number of hard links, permissions, ownership (UID/GID), and timestamps for last access, modification, change, and creation (birth time if supported).

If we break down:

|             |                                                           |
|-------------|-----------------------------------------------------------|
| File        | name of the file                                          |
| Size        | size in bytes                                             |
| Blocks      | number of filesystem blocks allocated                     |
| IO Block    | preferred block size for I/O operations                   |
| Device      | device identifier where the file resides                  |
| Inode       | unique identifier of the file in the filesystem           |
| Links       | number of hard links pointing to this inode               |
| Access      | file type and permission bits                             |
| Uid/Gid     | owner user and group                                      |
| Timestamps: |                                                           |
| Access      | last time file was read                                   |
| Modify      | last time content was changed                             |
| Change      | last time metadata (permissions, ownership, etc.) changed |
| Birth       | creation time (not always available on all filesystems)   |

So overall, `stat` is essentially a “full identity card” for a file in Linux.

We run `stat` on the symbolic link `file2`

```
[linux@fidir] stat file2
File: file2 -> file.txt
Size: 8      Blocks: 0      IO Block: 4096  symbolic link
Device: 259,2 Inode: 23364884  Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1000/  fidir)  Gid: ( 1000/  fidir)
Access: 2024-03-23 19:59:54.840037991 +0700
Modify: 2024-03-23 19:58:38.512035836 +0700
Change: 2024-03-23 19:59:46.536037757 +0700
Birth: 2024-03-23 19:58:38.512035836 +0700
```

The inode numbers are different because a symbolic link is treated as a separate file in the filesystem, with its own inode. A symbolic link acts as a pointer to another file, but it is still considered an independent filesystem object.

If inode 23364884 is removed, the symbolic link `file2` is deleted, while `file.txt` remains unchanged. Conversely, if inode 23364606 (the target file) is deleted, `file2` is not removed, but it becomes a broken (*dangling*) *symbolic link* that points to a non-existent file.

The `Links` field in the output does not refer to symbolic links; it refers to *hard links*. A *hard link* is a reference to a file that, unlike a *symbolic link*, is not a separate filesystem entity. Multiple hard links can point to the same *inode*, meaning a single inode number may correspond to multiple filenames.

For this reason, a symbolic link has its own *inode number*, different from the target file, while a *hard link* shares the same inode number as the original file.

We create a hard link to the file `file.txt` (command `ln` without options) and execute `stat`

```
[linux@fidir] ln file.txt hfile3
[linux@fidir] stat hfile3
File: hfile3
Size: 6      Blocks: 8      IO Block: 4096   regular file
Device: 259,2 Inode: 23364606  Links: 2
Access: (0644/-rw-r--r--)  Uid: ( 1000/   fidir)   Gid: ( 1000/   fidir)
Access: 2024-03-23 20:01:37.452040888 +0700
Modify: 2024-03-23 19:58:18.396035269 +0700
Change: 2024-03-23 20:08:06.860051880 +0700
Birth:   2024-03-23 19:58:11.664035079 +0700
```

The inode number of `hfile3` is the same as that of `file.txt`, and number of links is 2 because the same inode is associated with two filenames: `file.txt` and `hfile3`. However, if we execute `ls -l`, we can see that `hfile3` appears as a separate file and does not point to `file.txt`.

```
[linux@fidir] ls -l
total 8
lrwxrwxrwx 1 fidir fidir 8 Mar 23 19:58 file2 -> file.txt
-rw-r--r-- 2 fidir fidir 6 Mar 23 19:58 file.txt
-rw-r--r-- 2 fidir fidir 6 Mar 23 19:58 hfile3
```

In reality, `hfile3` is not a different file, but another reference to the same file. Even though it appears as a separate file in the output of `ls`, it is simply another filename used to access the same underlying data and inode.

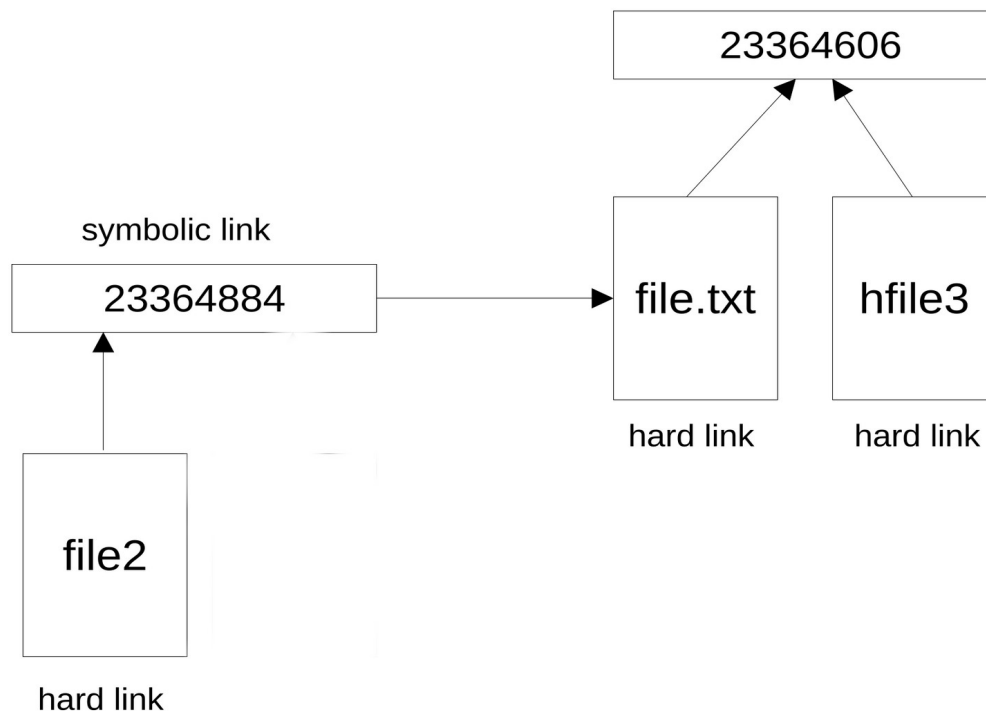
We add some text to `file.txt` and look at the content of `hfile3`

```
[linux@fidir] echo 'world' >> file.txt
[linux@fidir] cat hfile3
hello
world
```

When we write to `file.txt` we are effectively telling Linux: "Write this text into the file identified by inode number 23364606". Since this inode has two references (two filenames that refer to the same data), Linux updates the data associated with that inode, and both references will then access the updated content.

If we execute `cat` on the symbolik link `file2`, it will display the contents of `file.txt`. However, `file2` is not a direct reference to `file.txt` in the same way a hard link is. Instead, `file2` is a separate file whose inode contains a pointer to the path of another file. In other words, `file2` is itself a filesystem object (with its own inode), and that inode stores information that directs the operating system to `file.txt`. Therefore, unlike a hard link, a symbolic link does not share the same inode or data as the target file.

The following picture shows this relationship.



**Fig 1.** Relationship between hard links and symbolic links

`file2` is an hard link to the inode 23364884 that is a symbolic link to `file.txt` that is an hard link to the inode 23364606. `hfile3` is another hard link to the same inode. We can also create another hard link to the inode 23364884.

```

[linux@fidir] ln file2 sfile4
[linux@fidir] ls -l
total 8
lrwxrwxrwx 2 fidir fidir 8 Mar 23 20:38 file2 -> file.txt
-rw-r--r-- 2 fidir fidir 6 Mar 23 20:38 file.txt
-rw-r--r-- 2 fidir fidir 6 Mar 23 20:38 hfile3
lrwxrwxrwx 2 fidir fidir 8 Mar 23 20:38 sfile4 -> file.txt

```

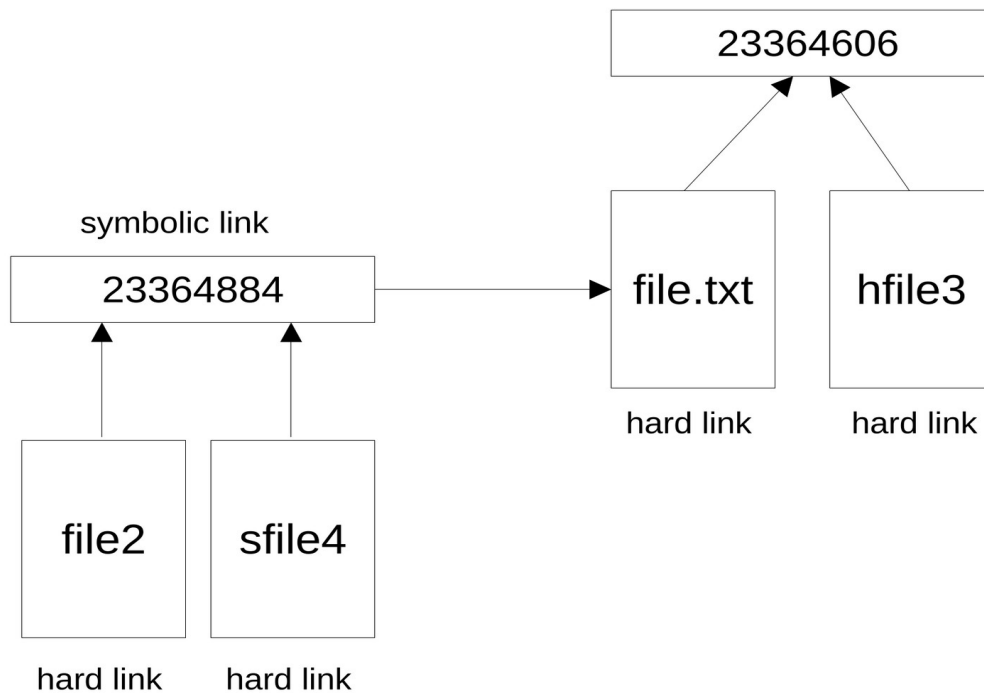
`sfile4` is another symbolic link to `file.txt`, this because we create an hard link that is a reference to an inode that points to another file (`file.txt`). We run `stat` on `sfile4`

```

[linux@fidir] stat sfile4
File: sfile4 -> file.txt
Size: 8      Blocks: 0      IO Block: 4096  symbolic link
Device: 259,2 Inode: 23364884  Links: 2
Access: (0777/lrwxrwxrwx)  Uid: ( 1000/  fidir)  Gid: ( 1000/  fidir)
Access: 2024-03-23 20:53:24.412128593 +0700
Modify: 2024-03-23 20:38:10.584102797 +0700
Change: 2024-03-23 20:53:22.808128548 +0700
Birth: 2024-03-23 20:38:10.584102797 +0700

```

The inode number is the same of `file2`. The following picture illustrates the current scenario.



**Fig 2.** Relationship between symbolic and hard links

`sfile4` (like `file2`) is an hard link to a symbolic link, or better to an inode that is a symbolic link. Both the inodes 23364606 and 23364884 represent a file, but the second represents a file that is a pointer to another file. A symbolic link has no content, is only a pointer to the data of another file.

We view the content af all created files.

```
[linux@fidir] cat *
hello
world
hello
world
hello
world
hello
world
```

We have four references to the same data with only two inodes.

The field `st_nlink` of the `stat` structure represents the number of the hard links of the file. The field `st_mode` defines type and mode of the file.

On a Linux system, files can be of different types:

#### *Regular file*

A file that contains data, it can be text or binary

#### *Directory*

An access point in the file system containing other files and directories



### Block file

A file that provides buffered I/O access to devices with a fixed-size units of data

### Character file

A file that provides unbuffered I/O access to devices with variable-size units of data

### FIFO

Used for communication between processes, also called *named pipe*

### Socket

Used for network communication between processes and for non-network communication between processes on the same host (*interprocess communication*)

### Symbolic link

A file that points to another file

These types are specified in the `st_mode` field as listed in the following table.

|                       |         |                                      |
|-----------------------|---------|--------------------------------------|
| <code>S_IFMT</code>   | 0170000 | bit mask for the file type bit field |
| <code>S_IFBLK</code>  | 0060000 | block device                         |
| <code>S_IFCHR</code>  | 0020000 | character device                     |
| <code>S_IFDIR</code>  | 0040000 | directory                            |
| <code>S_IFIFO</code>  | 0010000 | FIFO                                 |
| <code>S_IFLNK</code>  | 0120000 | symbolic link                        |
| <code>S_IFREG</code>  | 0100000 | regular file                         |
| <code>S_IFSOCK</code> | 0140000 | socket                               |

**Tab 4.** Values of `st_mode`

The program `getinfo.c` retrieves information about the file supplied on the command line<sup>8</sup>.

---

```
1 #include <stdint.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <sys/stat.h>
5 #include <sys/sysmacros.h>
6 #include <time.h>

7 int main (int argc, char *argv[])
8 {
9     struct stat sb;
10    if (argc != 2) {
11        fprintf(stderr, "Usage: %s <pathname>\n", argv[0]);
12        exit (EXIT_FAILURE);
```

---

<sup>8</sup> The code is taken from the `stat(2)` manual page

```

13     }
14     if (lstat(argv[1], &sb) == -1) {
15         perror("lstat");
16         exit (EXIT_FAILURE);
17     }
18     printf("ID of device: [%x, %x]\n", major(sb.st_dev), \
            minor(sb.st_dev));
19     printf("File type: ");
20     switch(sb.st_mode & S_IFMT) {
21         case S_IFBLK: printf ("block device \n"); break;
22         case S_IFCHR: printf ("character device \n"); break;
23         case S_IFDIR: printf ("directory \n"); break;
24         case S_IFIFO: printf ("FIFO/pipe \n"); break;
25         case S_IFLNK: printf ("symlink \n"); break;
26         case S_IFREG: printf ("regular file \n"); break;
27         case S_IFSOCK: printf ("socket \n"); break;
28         default: printf("unknown? \n"); break;
29     }
30     printf ("I-node number: %ju\n", (uintmax_t) sb.st_ino);
31     printf ("Mode: %jo (octal)\n", (uintmax_t) sb.st_mode);
32     printf ("link count: %ju\n", (uintmax_t) sb.st_nlink);
33     printf ("Ownership: UID=%ju GID=%ju\n", (uintmax_t) sb.st_uid, \
            (uintmax_t) sb.st_gid);
34     printf ("I/O block size: i %jd bytes\n", (intmax_t) sb.st_blksize);
35     printf ("File size: %jd bytes\n", (intmax_t) sb.st_blocks);
36     printf ("Block allocated: %jd\n", (intmax_t) sb.st_blocks);
37     printf ("Last status change: %s", ctime(&sb.st_ctime));
38     printf ("Last file access: %s", ctime(&sb.st_atime));
39     printf ("Last file modifications: %s", ctime(&sb.st_mtime));
40     exit (EXIT_SUCCESS);
41 }

```

---

### getinfo.c

At line 9, the program declares a variable of type `struct stat` named `sb`, which will store the file information returned by `lstat`, and at line 10, it checks whether exactly one pathname argument was provided on the command line.

At lines 11–12, if the pathname is missing, we print a usage message to `stderr` and terminates using `exit (EXIT_FAILURE)`.

At line 14, the program calls `lstat`, passing the pathname provided by the user and the address of the `sb` structure, where the file metadata will be stored. Next, at lines 15–16, if `lstat` fails, we print the corresponding error message using `perror` and terminates with `EXIT_FAILURE`.

At line 18, the program prints the major and minor numbers of the device containing the file by using the functions `major` and `minor`<sup>9</sup> defined in `sys/sysmacros.h`.

---

9 On Linux each device has a major and a minor number, the major number represents the device driver (IDE/SATA disk driver, floppy, serial port, etc..), the minor number identifies the device (many devices can share the same device driver)

At line 19, it starts to print the type of the file. The program performs a bitwise AND operation between `sb.st_mode` and the constant `S_IFMT`<sup>10</sup> at line 20, in order to extract the file type information from the `mode` field.

At lines 21–28, we use a `switch` statement to determine the file type and print the corresponding description, such as regular file, directory, symbolic link, socket, or device. Then, at line 30, we print the inode number stored in `sb.st_ino` after casting it to `uintmax_t`<sup>11</sup>.

At line 31, the file mode in octal format is printed after casting `sb.st_mode` to `uintmax_t`.

At line 32, the number of hard links associated with the file is printed using the field `sb.st_nlink`.

After printing the user ID (UID) and group ID (GID) that own the file at line 33, the program prints the preferred I/O block size for the filesystem and at line 35 prints the size of the file.

At line 36, it prints the number of allocated blocks used by the file.

At lines 37–39, it calls `ctime`<sup>12</sup> to convert the timestamps stored in `sb.st_ctime`, `sb.st_atime`, and `sb.st_mtime` from `time_t` format into human-readable ASCII strings, and then prints them.

We execute the program passing its source file.

```
[linux@fidir] ./getinfo getinfo.c
ID of device:          [103, 2]
File type:             regular file
I-node number:        23629952
Mode:                 100644 (octal)
link count:           1
Ownership:             UID=1000  GID=1000
I/O block size: i     4096 bytes
File size:             8 bytes
Block allocated:       8
Last staus change:    Sun Mar 24 22:06:09 2024
Last file access:     Sun Mar 24 22:06:29 2024
Last file modifications: Sun Mar 24 22:06:09 2024
```

We execute the program passing a device file.

```
[linux@fidir] ./getInfo /dev/tty0
ID of device:          [0, 5]
File type:             character device
I-node number:        13
Mode:                 20620 (octal)
link count:           1
Ownership:             UID=0  GID=5
I/O block size: i     4096 bytes
File size:             0 bytes
Block allocated:       0
Last staus change:    Wed Mar 20 17:57:16 2024
```

---

10 `S_IFMT` is the bitmask for the file type, AND-ing with `sb.st_mode` return the file type. It is defined in `sys/stat.h`

11 `intmax_t` and `uintmax_t` are the signed and signed types to represent any value of any integer type supported by the implementation, see the `intmax(3type)` manual page. It is used for the inodes because this can have larger values. These types are defined in `stdint.h`

12 `ctime` takes a pointer as argument, is defined in `time.h`

|                          |                          |
|--------------------------|--------------------------|
| Last file access:        | Wed Mar 20 17:57:16 2024 |
| Last file modifications: | Wed Mar 20 17:57:16 2024 |

The program successfully retrieved the `stat` information in both executions.

### Permissions

On Linux system (and Unix systems), each file has a set of permissions that define the access mode to the file.

| Type | User |   |   | Group |   |   | Other |   |   |
|------|------|---|---|-------|---|---|-------|---|---|
|      | r    | w | x | r     | w | x | r     | w | x |
|      | 4    | 2 | 1 | 4     | 2 | 1 | 4     | 2 | 1 |

**Fig 3.** File permissions

File permissions are divided into three categories: the owner (*user*) of the file, the *group* associated with the file, and *others* (all remaining users). The first field indicates the type of the file.

|   |   |                       |
|---|---|-----------------------|
| – | 1 | regular file          |
| d | 2 | directory             |
| l | 3 | symbolic link         |
| c | 4 | character device file |
| b | 5 | block device file     |
| p | 6 | FIFO (named pipe)     |
| s | 7 | socket                |

**Tab 5.** File types

The second field represents the permissions: the first set (3 elements) defines the *owner's permissions*, the next the *group's permissions*, the last the *others permissions*. Each permission has a numeric and a string value.

|   |         |   |
|---|---------|---|
| r | read    | 4 |
| w | write   | 2 |
| x | execute | 1 |

**Tab 6.** Types of permissions

*Read permission* means the user can read the contents of the file, *write permission* means the user can modify the file's contents, *execute permission* means the file can be executed. Concerning the directories: read means the user can view the directory's contents<sup>13</sup>, write is the ability to add and delete files in the directory<sup>14</sup>, execute means the user can traverse the directory (enter in the directory with the `cd` command)<sup>15</sup>. To add and delete files in a directory the user needs *execute* and *write permissions* on it.

The sum of these type specifies the permissions for the user/owner/other, so for example the owner can have a permission of 6 (6 = 4+2) that means read and write and no execute, the group can have a permission of 5 (5 = 4 + 1) that means read and execute but no write. All the possible combinations of 4,2 and 1 define a specific permission. The lowest permission is 000 that is defined by ---. Where there is no permission, Linux use a - to represent it. The highest permission is 777, where 7 is the sum of 4 + 2 + 1 which means read, write and execute.

The following table lists the possible combinations of values.

| Total | Sum       | rwX | Permission           |
|-------|-----------|-----|----------------------|
| 7     | 4 + 2 + 1 | rwx | read, write, execute |
| 6     | 4 + 2     | rw- | read and write       |
| 5     | 4 + 1     | r-x | read and execute     |
| 4     | 4         | r-- | read only            |
| 3     | 2 + 1     | -wx | write and execute    |
| 2     | 2         | -w- | write only           |
| 1     | 1         | --x | execute only         |
| 0     | 0         | --- | none                 |

**Tab 7.** Combinations of the permissions values

The following table lists some examples.

|     |                                                                                     |
|-----|-------------------------------------------------------------------------------------|
| 777 | user: read, write, execute; group: read, write, execute; other: read, write execute |
| 664 | user: red, write; group: read, write, other: read                                   |
| 755 | user: read, write, execute; group: read, execute; oher: read, execute               |
| 5-- | user: read, execute; group and others no permissions                                |
| -62 | user: no permission; group: read and write; other: write                            |

**Tab 8.** Example of permissions

To view the file permissions we can use `ls` with the option `-l`

<sup>13</sup> To see the files attributes (as shown by `ls -l`) the execution permission is required for the directory

<sup>14</sup> An user can delete a file in a directory if he has the write permission on that directory even if he is not the owner of that file

<sup>15</sup> To execute `cd /var/log` the user must have the execution permission for all the directories of the path: `/`, `var` and `log`

```
[linux@fidir] ls -l
total 8
lrwxrwxrwx 2 fidir fidir 8 Mar 23 20:38 file2 -> file.txt
-rw-r--r-- 2 fidir fidir 12 Mar 23 21:09 file.txt
-rw-r--r-- 2 fidir fidir 12 Mar 23 21:09 hfile3
lrwxrwxrwx 2 fidir fidir 8 Mar 23 20:38 sfile4 -> file.txt
```

`file.txt` and `hfile3` are regular file with 644, `file2` and `sfile4` are symbolic links with 777.

In the previous example we executed our program passing the file `/dev/tty`, the output for the `st_mode` field was

Mode: 20620 (octal)

The leading digit represents the file type/flags, followed by the standard permission values for owner, group, and others. In this case, the owner has permission value 6, the group has permission value 2, and others have permission value 0.

On Linux systems, a regular file is represented in the first bit of the permissions with a `-`.

The *permissions mask* is a bitmask of 10 bit where the first bit represents the type of the file and the others 9 bits represent the three-bit fields of the owner, group and others<sup>16</sup>. The field `st_mode` of the structure `stat` is a bit mask containing the permissions and the type of the file. The file's type can be extracted by ANDing the value of the field with the constant `S_IFMT`. In the `getInfo.c` program we compared this values with the constants to print the type of file. To perform this operation are also available the following macros defined in `sys/stat.h`.

| Macro                    | Value                                        | Description                                     |
|--------------------------|----------------------------------------------|-------------------------------------------------|
| <code>S_ISLNK(m)</code>  | <code>((m) &amp; S_IFMT) == S_IFLNK)</code>  | Returns true if the file is a symbolic link     |
| <code>S_ISREG(m)</code>  | <code>((m) &amp; S_IFMT) == S_IFREG)</code>  | Returns true if the file is a regular file      |
| <code>S_ISDIR(m)</code>  | <code>((m) &amp; S_IFMT) == S_IFDIR)</code>  | Returns true if the file is a directory         |
| <code>S_ISCHR(m)</code>  | <code>((m) &amp; S_IFMT) == S_IFCHR)</code>  | Returns true if the file is a character device  |
| <code>S_ISBLK(m)</code>  | <code>((m) &amp; S_IFMT) == S_IFBLK)</code>  | Returns true if the file is a block device      |
| <code>S_ISFIFO(m)</code> | <code>((m) &amp; S_IFMT) == S_IFIFO)</code>  | Returns true if the file is a FIFO (named pipe) |
| <code>S_ISSOCK(m)</code> | <code>((m) &amp; S_IFMT) == S_IFSOCK)</code> | Returns true if the file is a socket            |

**Tab 9.** Macro to retrieve the type of file

The program `getfiletype.c` retrieves the type of the file supplied on the command line.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <sys/stat.h>
4 int main (int argc, char *argv[])
```

<sup>16</sup> To change the permissions of a file see the `chmod(1)` manual page

```

5 {
6     struct stat sb;
7     if (argc != 2) {
8         printf("\nUsage: %s <file>", argv[0]);
9         exit(EXIT_FAILURE);
10    }
11    if (lstat(argv[1], &sb) == -1) {
12        perror("lstat");
13        exit (EXIT_FAILURE);
14    }
15    int type = sb.st_mode & S_IFMT;
16    if (S_ISREG(type))
17        printf("\n%s: Regular file", argv[1]);
18    else if(S_ISLNK(type))
19        printf("\n%s: Symbolic link", argv[1]);
20    else if(S_ISDIR(type))
21        printf("\n%s: Directory", argv[1]);
22    else if(S_ISCHR(type))
23        printf("\n%s: Character device", argv[1]);
24    else if(S_ISBLK(type))
25        printf("\n%s: Block device", argv[1]);
26    else if(S_ISFIFO(type))
27        printf("\n%s: FIFO", argv[1]);
28    else if(S_ISSOCK(type))
29        printf("\n%s: Socket", argv[1]);
30    return 0;
31 }

```

---

### getfiletype.c

At line 6, the program declares a variable of type `struct stat` named `sb`, which will store the file information returned by `lstat`. At line 11, the program calls `lstat` to retrieve information about the file specified on the command line and stores the result in the `sb` structure. At line 15, the program performs a bitwise AND operation between `sb.st_mode` and the constant `S_IFMT` to extract the file type information, and stores the result in the variable `type`. At lines 16–29, the program checks the file type using macros such as `S_ISREG`, `S_ISLNK`, `S_ISDIR`, and others, then prints the corresponding file type description.

We execute the program.

```
[linux@fidir] ./getfiletype getfiletype.c
getfiletype.c: Regular file
```

```
[linux@fidir] ./getfiletype /home
/home: Directory
```

```
[linux@fidir] ./getfiletype /usr/bin/X
/usr/bin/X: Symbolic link
```

```
[linux@fidir] ./getfiletype /tmp/dbus-5HHov05X
```

By using the `stat` structure, we can develop programs for many different purposes. For example, we can write a program that searches for files with specific permissions, files with a given last modification date, or files whose size is greater or smaller than a specified threshold, and many other similar criteria.

## 1.2 File permissions

In the previous section we introduced the permissions. The file's permissions are defined by the following constants in `sys/stat.h`.

| Constant             | Octal value | Permissions                   |
|----------------------|-------------|-------------------------------|
| <code>S_IRUSR</code> | 00400       | user read                     |
| <code>S_IWUSR</code> | 00200       | user write                    |
| <code>S_IXUSR</code> | 00100       | user execute                  |
| <code>S_IRWXU</code> | 00700       | user read, write and execute  |
| <code>S_IRGRP</code> | 00040       | group read                    |
| <code>S_IWGRP</code> | 00020       | group write                   |
| <code>S_IXGRP</code> | 00010       | group execute                 |
| <code>S_IRWXG</code> | 00070       | group read, write and execute |
| <code>S_IROTH</code> | 00004       | other read                    |
| <code>S_IWOTH</code> | 00002       | other write                   |
| <code>S_IXOTH</code> | 00001       | other execute                 |
| <code>S_IRWXO</code> | 00007       | other read, write and execute |

|                      |       |              |
|----------------------|-------|--------------|
| <code>S_ISUID</code> | 04000 | Set-user-ID  |
| <code>S_ISGID</code> | 02000 | Set-group-ID |
| <code>S_ISVTX</code> | 01000 | Sticky bit   |

**Tab 9.** File permissions

The naming convention of these constants uses letters that represent permission types. For example, `RWXU` indicates read, write, and execute permissions for the User, while `RUSR` refers to read permission for the User only, and `RWXG` represents read, write, and execute permissions for the Group, and so on.

To correctly apply these permissions when performing operations on files, the following rules must be considered:

- To open a file, execute permission is required on every directory included in the file's pathname



- The read and write permissions on a file determine whether it can be opened for reading or writing using the flags `O_RDONLY`, `O_WRONLY`, or `O_RDWR`
- To use the `O_TRUNC` flag when opening a file, write permission on that file is required
- To create a new file in a directory, both read and execute permissions on that directory are required; the same permissions are also needed to delete a file
- To execute a file (either script or binary), execute permission must be granted on the file

The last three permissions in the Table 9 represent:

### *SUID*

The *set-user-ID* bit allows users to execute a program with the privileges of the file's owner. In other words, when a program has SUID set, it runs as if the user executing it is the owner of the executable

### *SGID*

The *set-group-ID* bit has two main effects depending on context. On a directory, it ensures that all new files created inside inherit the group ownership of the directory rather than the user's primary group. On an executable file, it allows users to run the program with the privileges of the file's group owner

### *Sticky bit*

The *sticky bit* is used to restrict file deletion within a *shared directory*. When it is set, only the owner of the file, the owner of the directory, or the root user can delete or rename files within that directory, even if other users have write permissions on it

A listing of the `sudo`<sup>17</sup> program shows that the SUID bit is set, indicated by the `s` in the user (owner) execute permission field.

```
[linux@fidir] ls -l /usr/bin/sudo
-rwsr-xr-x 1 root root 306488 Jan 27 03:10 /usr/bin/sudo
```

`sudo` allows a permitted user to execute commands with elevated privileges, typically as the `root` user, as if they were the owner of those commands.

When a user runs `sudo`, the system temporarily grants them the permissions of the target user (by default, `root`), so the command is executed with those higher privileges rather than the user's normal ones.

A listing of the `wall`<sup>18</sup> program shows that the SGID bit is set, indicated by the `s` in the group execute permission field.

```
[linux@fidir] ls -l /usr/bin/wall
-rwxr-sr-x 1 root tty 39224 Jan 5 21:25 /usr/bin/wall
```

When a user executes `wall`, the message is sent to all terminals of users currently logged into the system. This is possible because the program has the SGID bit set, which makes it run with the privileges of the group owner of the executable.

<sup>17</sup> `sudo` allows user to execute programs as they were `root` (superuser), see the `sudo(1)` manual page

<sup>18</sup> `wall` is used to send message on the console to every user logged in the system

In this case, the program runs with the permissions of the `tty` group, allowing it to write to the terminal devices owned by that group. As a result, the user effectively acts as a member of the `tty` group while the program is running, enabling access to other users' terminals for broadcasting messages.

An `ls` of the root directory `/` shows that the `/tmp` directory has the sticky bit enabled.

```
[linux@fidir] ls /  
...output...  
drwxrwxrwt 68 root root 4096 Mar 25 18:50 tmp  
...output...
```

The `t` in the “others” permissions indicates that the directory has the sticky bit set. The `/tmp` directory allows all users to create files and normally also to delete them. However, when the sticky bit is enabled, this behavior changes: users can only delete files if they are:

- The owner of the file
- The owner of the directory
- The `root` user

All users can create files in the directory, but deleting a file is restricted and only allowed if one of the previously mentioned conditions is satisfied.

### 1.3 Process credentials and privileges

When a process is running, it operates with a set of privileges. The following table illustrates these privileges.

|                    |                                                |
|--------------------|------------------------------------------------|
| real user ID       | the user who executed the command              |
| real group ID      | the group of the user who executed the command |
| effective user ID  | the current user of the process                |
| effective group ID | the current group of the process               |

**Tab 10.** Processes privileges

The real user ID and the real group ID identify who we really are in the system. After the login we have both this value taken from the file `/etc/passwd`. The effective user ID and the effective group ID determine our file access permissions. Usually the effective user ID and the effective group ID are equal to the real user ID and the real group ID.

When a file has the set-user-ID (SUID) and/or set-group-ID (SGID) bits enabled and it is executed as a program, the process's effective user ID and/or effective group ID is set to match those of the file. For example, if a file is owned by the superuser (`root`) and has the SUID bit set, then when it is executed the resulting process runs with superuser privileges. In this case, the real user ID of the process remains unchanged, but the effective user ID is temporarily changed to `root`.

A listing of the file `/bin/passwd` shows that it has the set-user-ID enabled.

```
[linux@fidir] sudo ls -l /bin/passwd
-rwsr-xr-x 1 root root 72344 Feb  5 03:28 /bin/passwd
```

When a user changes their password, they execute the `passwd` command, which runs with elevated privileges. This is because the program typically has the SUID bit set, allowing it to run with superuser permissions. As a result, the effective user ID of the process becomes 0 (`root` UID). The effective user ID represents the identity a process uses to determine its access rights while it is running.

To prove this we open two shells, in the first shell we execute `passwd`, in the second one we run `ps`<sup>19</sup>.

```
[linux@fidir] passwd
Changing password for fidir.
Current password:
```

```
[linux@fidir] ps -eo pid,user,uid,euid,args | grep passwd
343289 root      0    0 passwd
343291 fidir    1000 1000 grep --color=auto passwd
```

The process `passwd` (PID 343289) is running as `root`.

The set-user-ID and the set-group-ID are contained in the `st_mode` field of the `stat` structure and can be tested against the constants `S_ISUID` and `S_ISGID`.

In addition to the privileges shown in Table 10, a process also has two additional identifiers: the *saved set-user-ID* and the *saved set-group-ID*. These values are preserved by the `exec` family of functions<sup>20</sup>.

#### *saved set-user-ID* (saved SUID)

Stores a copy of the *process's effective user ID* at the time an executable is launched. It allows a process to temporarily drop elevated privileges and later regain them if needed

#### *saved set-group-ID* (saved SGID)

Works in the same way, storing a copy of the *effective group ID*. It enables a process to temporarily change its group privileges and restore them later when required.

## 1.4 The file mode creation mask: `umask`

When a new file or directory is created, its user ID is set to the effective user ID of the process that created it, and its group ID is set to the effective group ID of that process.

The permissions of the newly created file or directory are then determined using the *umask*, which is a bitmask applied to the default permission set. It acts as a filter that removes specific permission

---

19 `ps` is the command to see the running processes, the option `e` allows to select all the processes, the option `o` allows to specify which fields must appear in the output, the `'| grep passwd'` command allows us to select only the string containing the word 'passwd' to be displayed from the output of `ps`. See the manual pages `ps(1)` and `grep(1)`

20 The `exec` functions are used to execute programs, see the `exec(3)` manual page

bits, ensuring that new files and directories are not created with more permissions than intended.

To see the *umask* value we can use the `umask` command.

```
[linux@fidir] umask
002

[linux@fidir] umask -S
u=rwx,g=rwx,o=rx
```

This value represents the permission bits that will be disabled for newly created files or directories. As a result, on a system with a *umask* of 022, new files or directories will have write permissions removed for the *group* and for *others*.

If the *umask* is 002, only the write permission for *others* is disabled, while *group* write permission is still allowed.

The `-S` option prints the value in symbolic format.

When creating files and directories the default value for the permissions are 0666 for files and 777 for directories.

```
[linux@fidir] touch newregular.txt

[linux@fidir] ls -l newregular.txt
-rw-rw-r-- 1 fidir fidir 0 May  8 20:43 newregular.txt
```

Since the current *umask* is 002, the `newregular.txt` file has permissions set to 664.

$666 \& \sim 002 = 664$  (`-rw-rw-r--`)

The Standard C Library provides the `umask`<sup>21</sup> function to set the *file mode creation mask*.

```
#include <sys/stat.h>

mode_t umask(mode_t mask);

Returns: the previous value of the mask
```

The `umask` function sets the calling process's file mode creation mask (*umask*) to `mask & 0777`. It performs a bitwise AND operation between the provided `mask` and 0777, ensuring that only the permission bits for the owner, group, and others are considered.

As a result, the bits specified in the `mask` are effectively disabled (turned off) when new files or directories are created, preventing those permissions from being granted by default. A call to `umask` returns the previous value of the mask.

The program `mask.c` disables the *group* and *other* permission bits.

```
1 #include <stdlib.h>
2 #include <sys/stat.h>
```

---

21 See the `umask(2)` manual page

```

3 #include <fcntl.h>

4 #define RWRWRW (S_IRUSR | S_IWUSR | S_IRGRP | S_IWGRP | S_IROTH | S_IWOTH)

5 int main()
6 {
7     umask(S_IRGRP | S_IWGRP | S_IROTH | S_IWOTH);
8     creat("newfile.txt", RWRWRW);
9     return 0;
10 }

```

---

### mask.c

At line 4 we define the constant RWRWRW with the value 666 (rw-rw-rw). At line 7 we call `umask` passing the bits we want to disable (read and write for *group* and *others*). At line 8 we create a file using `creat` passing RWRWRW as permissions. Since `rw` for *group* and *others* were disabled by the previous call to `umask`, the file permissions will have a value of 600.

We execute the program.

```

[linux@fidir] ./mask

[linux@fidir] ls -l
-rw----- 1 fidir fidir  0 Mar 26 11:12 newfile.txt

```

The file `newfile.txt` has been created with the permissions 0600.

The `umask` call sets the *umask* value to 066 and when `creat` is called the requested mode is RWRWRW = 666. Since *umask* is set to 066, the final permissions of the file become:

$0666 \& \sim 0066 = 0600$

Let's suppose we want to create a file with the *group* and *others* to have the read permissions:

```
umask(S_IWGRP | S_IWOTH);
```

Or if we want to have the execution permission for the owner:

```

#define FILEPERM (S_IRUSR | S_IWUSR | S_IXUSR | S_IRGRP | S_IWGRP |
S_IROTH          | S_IWOTH)

```

The `umask` function takes as its argument either permission constants (such as `S_IRGRP`, `S_IWGRP`, etc.) or an octal value representing the mask. Another example of its use with the `open` function is shown in the program `mask2.c`.

---

```

1 #include <sys/stat.h>
2 #include <fcntl.h>

3 #define FILEPERM (S_IRUSR | S_IWUSR | S_IRGRP | S_IWGRP | S_IROTH)
4 #define NOPERM (S_IRGRP | S_IWGRP | S_IROTH | S_IWOTH)

```

```

5 int main()
6 {
7     umask(0);
8     int f = open("newfile.txt", O_CREAT | O_RDWR | O_EXCL, FILEPERM);
9     umask(NOPERM);
10    int f2 = open("newfile2.txt", O_CREAT | O_RDWR | O_EXCL, FILEPERM);
11    return 0;
12 }

```

---

### **mask2.c** Change the mask and creates two file

At line 3 we define the `FILEPERM` constant with a set of permissions and at line 4 the `NOPERM` constant with permissions to turn off. We set `umask` to 0 (don't turn off any bits, the file will be created exactly with the permissions passed to `open`) at line 7. At line 8 we open the file (creating it) passing the permissions defined in `FILEPERM`. We set the `umask` to disable the bits specified in `NOPERM` and then we create another file using the new mask.

We execute the program.

```

[linux@fidir] ./mask2

[linux@fidir] ls -l
-rw-rw-r-- 1 fidir fidir 0 Mar 26 13:46 newfile.txt
-rw----- 1 fidir fidir 0 Mar 26 13:46 newfile2.txt

```

Since `FILEPERM` is 664 and `umask` is 0, `newfile.txt` has been created with permissions 664. After setting the `umask` to `NOPERM`, which is 066, `newfile2.txt` has been created with permissions 600.

The octal values used with `umask` can be specified as a single value or as a combination (sum) of multiple values from the available permission bit set.

| Mask bit | Permission    |
|----------|---------------|
| 0400     | user-read     |
| 0200     | user-write    |
| 0100     | user-execute  |
| 0040     | group-read    |
| 0020     | group-write   |
| 0010     | group-execute |
| 0004     | other-read    |
| 0002     | other-write   |
| 0001     | other-execute |

**Tab 11.** `umask` file access permission bits

The `umask` command can show the *umask* value in both the format: octal and string. It can also be used to change the current value.

```
[linux@fidor] umask 027
```

```
[linux@fidor] umask
```

```
027
```

```
[linux@fidor] umask -S
```

```
u=rwx,g=rx,o=
```

A process that calls `umask` to change its mask does not affect the shell's *umask* value. The shell's *umask* is set at login and remains unchanged for the duration of the session unless it is explicitly modified using the `umask` command.